

Machine Learning Training Methods

A Quick Reference Guide

Prepared by Nikhil Reddy Kethireddy | AIML-500, Indiana Wesleyan University | Condensed from a guided AI tutoring session on how models learn.

How to use this guide

This is a one-sitting refresher, not a textbook. Use it three ways: **(1)** before a meeting where someone will pitch a machine learning solution, to check whether the problem actually has labels; **(2)** when scoping a project, to decide where the effort should go before anyone writes code; **(3)** as a translation layer when you need to explain a model's limits to a non-technical stakeholder — the everyday comparisons in the tables are there to be borrowed.

1. The three ways a model learns

Method	How it learns	What it needs	Everyday comparison
Supervised	Guesses, compares its guess to the correct answer, then adjusts to shrink the error. Repeats.	Labelled data — every input paired with a known correct output.	A student with a teacher who has the answer key.
Unsupervised	No answer key to check against, so it groups similar items by their features (clustering).	Unlabelled data, and features that actually distinguish things.	Sorting a pile of mixed LEGO bricks with no instructions — you sort by colour, size, shape.
Reinforcement	Acts, receives a reward or penalty, updates its strategy to maximise total reward over time.	An environment to act in and a reward signal worth optimising.	Training a dog — the treat teaches which behaviour to repeat.

The one question that picks the method: do I already have the right answers written down? Yes → supervised. No, and I want structure → unsupervised. No, but I can score outcomes → reinforcement.

2. The training workflow

#	Step	What happens
1	Collect	Gather the raw material — text, images, numbers, records.
2	Prepare	Clean and format it: missing values, inconsistent units, normalisation. Usually the longest step.
3	Choose a model	Pick the algorithm suited to the job and the size of the data.
4	Train	Feed the data in so the model can find patterns.
5	Evaluate	Test against data it has never seen. Performance on training data proves nothing.
6	Tune	Adjust settings and repeat.

Why the data gets passed through more than once (epochs): a single pass lets the model latch onto coincidence. Repeated passes let it correct earlier errors and settle on the patterns that hold across examples rather than memorising individual rows.

3. What each piece of the system actually does

Piece	Role	Kitchen equivalent
Algorithm	The instructions — the order of operations for learning.	The recipe

Data and features	The raw material, and the specific attributes drawn out of it.	The ingredients
Compute	The processing power that does the work.	The oven

The distinction matters in budget conversations. Buying a faster oven does not fix bad ingredients.

4. Data quality: the three things to check first

- **Quality.** Errors, gaps, and bias in the data get learned as if they were true. Garbage in, garbage out.
- **Quantity.** Too few examples and the model cannot separate a real pattern from a coincidence.
- **Relevance.** The data has to represent the problem. A model shown only dogs will never learn what a cat is.

The trap that catches real projects — label quality. On any supervised task built from historical records, the labels were applied by people. A help desk classifier trained on thousands of past tickets learns whatever the agents typed, including their habitual mistakes: if billing issues were routinely filed under technical support, the model reproduces that error faithfully and confidently. The model does not know the truth; it knows the labels. Before training, fix the labels — or have several people tag the same records and use the consensus.

5. Better features or more data?

The honest answer is that it depends on scale. More data tends to win when the features are adequate and the volume is large enough for patterns to emerge anyway. Better features tend to win when the dataset is small, because every attribute has to carry weight.

On a small budget, invest in features. More data at small scale often adds noise rather than signal, and feature work can be done in-house — it does not require paying a labelling team. Feature engineering stretches the value of the data already in hand.

6. Why a reinforcement agent accepts a bad move

The goal is **cumulative** reward, not the immediate one — which is why an agent will take a worse move now to set up a better outcome later.

It can tell the sacrifice was worth it because it keeps a running value score for situations, not just actions. When the reward finally arrives, credit travels backwards: the states that made the win reachable have their value raised, even though no reward was collected at the time. In chess terms, losing the queen looks terrible on that move and turns out to be the move that made the checkmate possible ten moves later.

7. Verifying what an AI tutor tells you

An AI assistant predicts likely responses from patterns; it does not know facts the way a person does, and it can be confidently wrong. Standard, well-established material is usually reliable — but reliability should be checked, not assumed.

- **Cross-reference.** Anything surprising gets looked up in a textbook or a reputable source.
- **Test it.** Where possible, build the simple version yourself and see whether the process matches the description.
- **Ask why.** Follow the reasoning rather than the conclusion. If the logic does not hold, push back.

Source: guided tutoring session with the AIML-500 Machine Learning Training Methods Coach (SchoolAI), July 2026, with supporting reading from *A Beginner's Guide to Artificial Intelligence and Machine Learning*. Examples and the scoping judgements in sections 4 and 5 are my own, drawn from IT service management work.